[승코의 차근차근 Flutter] 03. 다트 비동기 프로그래밍

📍 3.1 동기 vs 비동기

동기(Synchronous)	비동기(Asynchronous)
한 줄씩 순차적으로 실행	오래 걸리는 작업은 "기다리지 않고" 다음 코드 실행
앞 코드가 끝나야 다음 코드 실행됨	기다릴 동안 다른 작업을 먼저 처리함
느림 / CPU 낭비 가능	효율적 / 빠른 사용자 경험

예: 음식 주문 비유

- **동기**: 음식점에서 주문하면 요리 다 끝날 때까지 그 자리에서 기다림.
- **비동기**: 음식 주문하고 벨 받아서 다른 일 하고 있다가 벨 울리면 음식 받으러 감.

📍 3.2 Future: 미래에 결과를 가져올 객체

```
Future<String> name;  // 미래에 String을 줄거야
Future<int> number;   // 미래에 int를 줄거야
```

> Future는 "결과를 나중에 줄게!"라는 약속

Future.delayed()

시간이 걸리는 작업을 흉내 낼 수 있는 함수

```
Future.delayed(Duration(seconds: 3), () {
  print('3초 뒤 실행됨!');
});
```

이걸 함수 안에서 써보면?

```
void addNumbers(int a, int b) {
  print('계산 시작!');
  Future.delayed(Duration(seconds: 3), () {
    print('${a + b}');
  });
  print('계산 끝!');
}
```

🧠 출력 순서:

```
계산 시작!
계산 끝!
3초 뒤에: 2
```

> "3초 기다리는 동안 다음 코드(계산 끝!) 먼저 실행됨"

📍 3.3 async / await

- `async`: 이 함수는 비동기야! 라고 표시
- `await`: 결과 나올 때까지 기다려줘!

```
Future<void> addNumbers(int a, int b) async {
  print('계산 시작!');
  await Future.delayed(Duration(seconds: 3), () {
    print('${a + b}');
  });
  print('계산 끝!');
}
```

이제 출력 순서는?

```
계산 시작!
3초 뒤: 2
```

```
계산 끝!
```

🧠 왜?

- `await` 가 “딱 여기서 기다려줘!”라고 명령했기 때문

📍 3.3.1 반환값이 있는 async 함수

```
Future<int> addNumbers(int a, int b) async {
  await Future.delayed(Duration(seconds: 1));
  return a + b;
}
```

```
void main() async {
  int result = await addNumbers(1, 2);
  print('결과: $result');
}
```

> await는 결과를 받아와서 변수에 저장할 수 있다. 완전 일반 함수처럼 사용 가능!

📍 3.4 Stream: 여러 값을 "계속" 받아오는 비동기

> Future는 "딱 한 번" 값을 줌
>
> Stream은 "계속" 값을 줌 (ex: 채팅, 센서 데이터, 실시간 데이터 등)

```
import 'dart:async';

void main() {
  final controller = StreamController();
```

```
  final stream = controller.stream;

  // 스트림을 듣기 시작
  stream.listen((val) {
    print('받은 값: $val');
  });

  // 스트림에 값 추가
  controller.sink.add(1);
  controller.sink.add(2);
}
```

📍 3.4.2 브로드캐스트 스트림

> 기본 Stream은 한 번만 listen 가능
>
> 여러 리스너가 동시에 듣게 하고 싶을 땐?

```
final stream = controller.stream.asBroadcastStream();

stream.listen((val) => print('리스너1: $val'));
stream.listen((val) => print('리스너2: $val'));
```

→ `controller.sink.add(1)` 할 때

두 리스너가 **동시에** 값을 받는다!

📍 3.4.3 Stream을 함수처럼 만들기

> Future와 비교

- `Future`는 `async`, `return`
- `Stream`은 `async*`, `yield`

```
Stream<String> countUp() async* {
  for (int i = 0; i < 5; i++) {
    yield '현재 숫자: $i';
    await Future.delayed(Duration(seconds: 1));
  }
}
```

```
void main() {
  countUp().listen((val) {
    print(val);
  });
}
```

출력:

```
현재 숫자: 0
현재 숫자: 1
...
```

✅ 정리 한눈에 보기

개념	키워드	특징
동기	없음	한 줄씩 순서대로 실행
비동기 (Future)	`Future`, `await`, `async`	결과를 나중에 줌 (한 번만)
스트림 (Stream)	`Stream`, `async*`, `yield`	여러 값을 시간에 따라 계속 줌
async	함수 선언할 때	비동기 함수임을 표시
await	비동기 호출 앞에	결과 나올 때까지 기다림
yield	Stream 함수에서	하나씩 값을 계속 줌